CPSC370 · AGENTIC AI SYSTEMS · FINAL PROJECT

Chapman Agentic Scheduling Challenge

Specification version 1.1 · November 2026

What changed in v1.1. Version 1.1 adds everything needed to work with the **provost orchestrator**, the university coordinator your agent will talk to. The required `run_agent(request) -> dict` interface, the three operations, and the response formats are **unchanged**. A v1.0-compliant agent stays compliant. The additions are:

- how your agent connects (`connect.py`, no server needed)
- the local Docker practice environment
- the full resource envelope included in every request
- the new resource fields and constraint types
- the disruption formats
- time-overlap semantics
- timeouts
- how the provost measures your metrics
- how historical evidence reaches your agent, and the `input_samples/` folder

Sections marked NEW or UPDATED carry these changes. All code and data are in the course repository: `github.com/erikexpertai/cpsc370_agentic_final`.

Project Overview

In this project, each team will design an LLM-based scheduling agent for one of Chapman University's 11 schools and colleges. Your agent will receive a set of available meeting times and classroom allocations from an instructor-provided university scheduling coordinator, the **provost orchestrator**. Using those resources, your agent must create a feasible schedule for its academic unit, explain its choices, respond to proposed schedule changes, and repair its schedule when resources or constraints change.

All 11 academic-unit agents will be integrated at the end of the semester with a common coordinator. The final system will therefore be a class-built multi-agent scheduling environment: each unit controls its own local scheduling decisions while the coordinator manages university-wide conflicts and requests changes.

The project is intentionally designed so that strong programming is not required. You may use any contemporary LLM agent framework, a simple Python wrapper, prompt/configuration files, retrieval over historical schedule data, or a combination of approaches. What matters is whether your agent behaves correctly, uses evidence and constraints appropriately, and conforms exactly to the common interface described in this specification.

Academic Units

Each team will be assigned one of the following academic units. The **unit code** is what you pass to the connector. NEW

Code	Academic unit	Code	Academic unit
SBE	Argyros College of Business and Economics	SENG	Fowler School of Engineering
CES	Attallah College of Educational Studies	SOL	Fowler School of Law
COPA	College of Performing Arts	COS	Schmid College of Science and Technology
CHBS	Crean College of Health and Behavioral Sciences	SOC	School of Communication
CFMA	Dodge College of Film and Media Arts	SOP	School of Pharmacy
CHSS	Wilkinson College of Arts, Humanities, and Social Sciences		

The 11-unit list follows Chapman's current schools-and-colleges structure. Your team represents the academic unit as a whole, not a single department.

What the Coordinator Provides UPDATED

Your agent does not choose the university-wide room inventory or time grid. The coordinator provides the **resource envelope** your academic unit may use:

- A list of courses/sections that your academic unit must schedule.
- A list of available meeting times (a canonical university grid of about 29 slots).
- A list of classrooms allocated or available to your unit, including **shared rooms** that other units also hold.
- Room capacity and relevant room features.
- Current faculty/course constraints supplied for the scheduling run.

- Any university-wide hard constraints that apply to all agents.

The full envelope is included in every request (`generate_schedule`, `evaluate_change` and `repair_schedule`), so your agent can be stateless. The envelope can **change during a run**: disruptions and coordinator reservations are written into it. Always use the envelope in the current request, never a cached copy.

Where conflicts come from. About 44% of Chapman classrooms were historically used by more than one school, some instructors teach in more than one unit, and some courses are cross-listed between units. The coordinator gives shared rooms to every unit that uses them, so units will collide. Resolving those collisions is the multi-agent part of the project.

What Your Agent Decides

- Which course is assigned to which available time.
- Which allocated classroom is assigned to each course.
- How to resolve competing preferences within your unit.
- Which proposed schedule changes are acceptable.
- Which alternatives to suggest when a proposed change is unacceptable.
- How to repair the unit schedule when a room, time, faculty availability, or other resource changes.

Use of Historical Schedule Data UPDATED

You may use historical Chapman schedule data in any way you believe improves your agent. Historical data is optional and advisory; it must never override current hard constraints.

- Use prior schedules directly as examples in the agent prompt.
- Retrieve similar historical courses or schedules at runtime.
- Compute empirical preferences, such as common meeting days or times for a course or faculty member.
- Infer common room types or scheduling patterns.
- Use historical schedules to construct few-shot examples.
- Ignore historical data entirely and rely on explicit current constraints.

If you use historical data, document exactly how it is used and identify any limitations. Copying a previous semester's schedule without reasoning about the current resource allocation is not sufficient.

HISTORICAL EVIDENCE IN THE PROVOST'S DATA

The full registrar schedule is **not** distributed. Historical practice reaches your agent through the envelope:

- Each course's `preferred_times` is the slot its section historically met in, snapped to the current grid.
- Each instructor's `preferred_times` collects the slots they historically taught in.
- Room `features` are inferred from what historically met in each room.

Instructors appear only as pseudonyms (`FAC-xxxxxx`), and the same pseudonym is used in practice and on integration day. If you want more history, you can build your own: record the requests from practice runs with `--save-dir`, or use public sources you are authorized to use.

Scheduling Rules UPDATED

Your agent must distinguish between hard constraints and soft preferences. Hard constraints determine whether a schedule is valid. Soft preferences determine whether one valid schedule is better than another.

HARD CONSTRAINTS

- A course must use one of the time slots supplied by the coordinator.
- A course must use one of the rooms supplied or allocated to the unit.
- Two courses may not occupy the same room at the same time.
- One instructor may not teach two courses at the same time.
- Expected enrollment may not exceed room capacity.
- A course requiring a specialized room feature must be assigned to a room with that feature.
- Explicit course-pair conflict constraints supplied in the input must be respected.
- Any additional hard constraints included in the request must be respected. In Protocol 1.0 these are:
  - room `unavailable_times`
  - faculty `unavailable_times`
  - `blackout` windows
  - `forbidden_timeslots`

  These are described under Protocol Details below.

"At the same time" means overlapping, not the same ID. Two timeslots conflict if they share at least one day and their clock intervals overlap, even when their `timeslot_id`s differ. For example, `MWF-1000` (10:00–10:50) overlaps `MW-1000` (10:00–11:15). Day codes are `M T W R F`, where `R` is Thursday.

A response containing an avoidable hard-constraint violation is considered invalid, even if the schedule is otherwise attractive. The provost re-checks every schedule you return. Entries that violate a hard constraint are dropped, and those courses count as unresolved.

SOFT PREFERENCES

Soft preferences may differ by academic unit and may be informed by historical schedules. Examples include:

- Faculty preferred teaching days or times.
- Avoiding early morning, late evening, or Friday sections.
- Keeping required courses from the same program separated.
- Minimizing long gaps in a faculty member's teaching day.
- Using historically common meeting patterns.
- Keeping a course in a preferred building or room type.
- Minimizing changes when repairing an existing schedule.
- Preferring rooms marked `"shared": false`, which other units can't contest.

Your team decides how to represent and prioritize soft preferences. You may use a weighted scoring function, LLM reasoning, rules, an optimizer, or a hybrid approach. You must be able to explain your choice.

Required Agent Capabilities

Every academic-unit agent must support all three operations below.

Operation	Purpose	Required behavior
`generate_schedule`	Create a schedule from the supplied resource envelope.	Return a complete proposed local schedule plus rationale and unresolved issues.
`evaluate_change`	Evaluate a proposed change to one scheduled course.	Accept or reject the change, explain why, and provide valid alternatives when rejecting.

Operation	Purpose	Required behavior
`repair_schedule`	Repair an existing schedule after a disruption.	Produce a new valid schedule while minimizing unnecessary changes where possible.

Technical Freedom

You may implement the internals of your agent however you want, provided it follows the required interface. Examples include:

- A simple Python program that calls an LLM.
- OpenAI Agents SDK, LangGraph, CrewAI, PydanticAI, Google ADK, Microsoft Agent Framework, or another agent framework.
- A prompt-driven agent with configuration files.
- Retrieval-Augmented Generation (RAG) over historical schedules or policy documents.
- A hybrid system combining deterministic constraint checks with LLM reasoning.
- A mathematical optimizer used as a tool by an LLM agent.

Using a sophisticated framework does not automatically earn a higher grade. A simple system that is reliable, explainable, and well evaluated may outperform a complex system.

Required Integration Interface

All teams must conform to the same interface so the university coordinator can call every academic-unit agent without knowing how it is implemented.

REQUIRED PYTHON ENTRYPOINT

Your submission must expose the following callable from `agent.py`:

```
def run_agent(request: dict) -> dict:
    """Process one coordinator request and return a JSON-compatible dictionary."""
    ...
```

The instructor starter code (`student_kit/connect.py`) handles connecting to the coordinator. Your LLM credentials stay on your own machine. The provost never sees them. Teams using a framework may wrap their framework inside `run_agent()`. Load prompts, configuration and data relative to your agent's own file (`Path(__file__).parent`), not the current working directory.

COMMON REQUEST ENVELOPE

```
{
  "protocol_version": "1.0",
  "request_id": "req-001",
  "operation": "generate_schedule",
  "unit": "Fowler School of Engineering",
  "semester": "Fall 2027",
  "payload": { ... }
}
```

Required fields:

- **protocol_version**: must be "1.0".
- **request_id**: unique identifier supplied by the coordinator; return it unchanged.
- **operation**: one of generate_schedule, evaluate_change, or repair_schedule.
- **unit**: the academic unit your agent represents (full name, as in the table above).
- **semester**: scheduling context.
- **payload**: operation-specific input described below.

COMMON RESPONSE ENVELOPE

```
{
  "protocol_version": "1.0",
  "request_id": "req-001",
  "unit": "Fowler School of Engineering",
  "status": "completed",
  "result": { ... },
  "warnings": [],
  "errors": []
}
```

- **status** must be completed, partial, or failed.
- **result** contains operation-specific output.
- **warnings** contains non-fatal issues or uncertainty.
- **errors** contains fatal problems that prevented completion. It must be non-empty when status is failed.
- The response must be JSON-serializable. Do not return framework-specific objects.

Operation 1: generate_schedule

Request payload (abridged; a full example is in `input_samples/01_generate_schedule.request.json`):

```
{
  "courses": [
    {
      "course_id": "CPSC-350-01",
      "title": "Data Structures and Algorithms",
      "instructor_id": "FAC-014",
      "expected_enrollment": 34,
      "required_features": ["computer"],
      "preferred_times": ["MW-1030", "TR-1000"],
      "avoid_times": ["MW-0800"],
      "conflicts_with": ["CPSC-380-01"]
    }
  ],
  "timeslots": [
    { "timeslot_id": "MW-1030", "days": ["M", "W"], "start": "10:30", "end": "11:45" }
  ],
  "rooms": [
    { "room_id": "SWE-150", "capacity": 40, "features": ["computer"],
      "building": "SWE", "shared": true, "unavailable_times": [] }
  ],
  "faculty_constraints": [
    { "instructor_id": "FAC-014", "unavailable_times": ["TR-1300"], "preferred_times":
["MW-1030"] }
  ],
  "global_constraints": [],
  "context": { "unit_code": "SENG" }
}
```

Required result:

```
{
  "schedule": [
    {
      "course_id": "CPSC-350-01",
      "timeslot_id": "MW-1030",
      "room_id": "SWE-150",
      "rationale": [
        "Room capacity satisfies expected enrollment.",
        "Room provides required computing feature.",
        "Time is preferred by the instructor."
      ],
      "preference_score": 0.91
    }
  ],
  "unresolved": [],
  "summary": "All courses scheduled with no hard-constraint violations."
}
```

`preference_score` must be a number from 0.0 to 1.0 representing how strongly the agent prefers the selected assignment relative to other feasible choices. You decide how to compute or estimate this score, but document your method.

Unresolved items may be course-ID strings or objects of the form `{"course_id": ..., "reason": ...}`. Objects are preferred because they explain why. Every course in the request must appear either in `schedule` or in `unresolved`. The provost counts a course that appears in neither as **silently dropped**.

Operation 2: evaluate_change UPDATED

The coordinator uses this operation when it needs your unit to consider moving one course because of a university-wide conflict. The proposal is always free of conflicts with the *other* units. Your job is to judge it against your own unit's constraints and preferences.

```
{
  "current_schedule": [ ... ],
  "proposed_change": {
    "course_id": "CPSC-350-01",
    "new_timeslot_id": "TR-1000",
    "new_room_id": "SWE-150"
  },
  "reason": "Resolve shared room SWE-150: SBE ECON-201-02 holds MW-1030/SWE-150.",
  "courses": [ ... ], "timeslots": [ ... ], "rooms": [ ... ],
  "faculty_constraints": [ ... ], "global_constraints": [ ... ], "context": { ... }
}
```

The `reason` names the conflict type and the other unit's course that keeps the contested resource. The conflict types are `shared room`, `shared instructor` and `cross list mismatch`.

Required result:

```
{
  "decision": "reject",
  "reason": "The proposed time conflicts with another required course.",
  "impact_score": 0.82,
  "alternatives": [
    {
      "timeslot_id": "MW-1300",
      "room_id": "SWE-150",
      "impact_score": 0.22,
      "reason": "No hard conflicts and only a minor preference penalty."
    }
  ]
}
```

- **decision** must be accept or reject.
- **impact_score** must be between 0.0 and 1.0, where 0.0 means essentially no disruption and 1.0 means severe disruption or infeasibility.
- If you reject a proposal and at least one valid alternative exists, return one or more alternatives.
- A rejection must identify the specific constraint or preference that caused the rejection.
- **An alternative must move the course.** Offering the course's current placement, which is the one in conflict, is not an alternative. Alternatives that reuse the contested resource at an overlapping time can't be used by the provost.
- **Accept is not a commit.** The provost decides and commits changes. You see the outcome in the next request's `current_schedule`.

Operation 3: repair_schedule UPDATED

The coordinator uses this operation after a disruption, such as a room becoming unavailable, a faculty constraint changing, or enrollment increasing.

```
{
  "current_schedule": [ ... ],
  "disruptions": [
    { "type": "room_unavailable", "room_id": "SWE-150" }
  ],
  "timeslots": [ ... ],
  "rooms": [ ... ],
  "courses": [ ... ],
  "faculty_constraints": [ ... ],
  "global_constraints": [ ... ],
  "context": { ... }
}
```

The envelope in the request **already reflects** the disruption. For example, a closed room has been removed from `rooms`. The `disruptions` list tells you *why* things changed. Disruption types:

type	fields	effect on the envelope
`room_unavailable`	`room_id`, optional `timeslot_ids`	Without `timeslot_ids` the room is removed. With them, they are added to the room's `unavailable_times`.
`room_capacity_change`	`room_id`, `new_capacity`	The room's `capacity` is updated.
`faculty_unavailable`	`instructor_id`, `timeslot_ids`	The slots are added to the instructor's `unavailable_times`.

type	fields	effect on the envelope
`enrollment_change`	`course_id`, `new_enrollment`, optional `unit`	The course's `expected_enrollment` is updated.
`blackout`	`days`, `start`, `end`	A `blackout` entry is added to `global_constraints`.
`course_must_move`	`course_id`, `forbidden_timeslot_ids`	A `forbidden_timeslots` entry is added to `global_constraints`.

Required result:

```
{
  "schedule": [ ... ],
  "changes": [
    {
      "course_id": "CPSC-350-01",
      "old_timeslot_id": "MW-1030",
      "old_room_id": "SWE-150",
      "new_timeslot_id": "MW-1030",
      "new_room_id": "SWE-201",
      "reason": "Original room unavailable."
    }
  ],
  "change_count": 1,
  "unresolved": [],
  "summary": "Schedule repaired with one course moved."
}
```

The repair objective is not simply to produce any new schedule. Your agent should preserve as much of the existing schedule as possible while restoring feasibility. `change_count` must equal the length of `changes`. The provost measures stability by comparing your returned schedule with `current_schedule`, not by trusting `change_count`.

Escalations. When a university-wide conflict can't be resolved by moving a single course, the provost sends an ordinary `repair_schedule` carrying a reservation: a `room_unavailable` or `faculty_unavailable` with specific `timeslot_ids`, or a `course_must_move` that forbids every slot but one. Your repair may need to move *other* courses to satisfy it.

Protocol Details NEW

ADDITIONAL ENVELOPE FIELDS

Field	Meaning
`rooms[].unavailable_times`	**Hard.** The room can't be used at any slot that overlaps one of these slots.
`rooms[].shared`	Other units also hold this room, so it may be contested.
`rooms[].building`	Informational.
`faculty_constraints[].unavailable_times`	**Hard.** Overlap semantics apply.
`context`	Unit code and data-provenance notes. Informational.

GLOBAL_CONSTRAINTS TYPES

```
{"type": "blackout", "days": ["W"], "start": "12:00", "end": "13:00"}
{"type": "forbidden_timeslots", "course_id": "CPSC-350-01", "timeslot_ids": ["MW-1030"]}
{"type": "cross_list_match", "rule": "same_timeslot", "owner": "<unit name>",
 "courses": [{"unit": "<unit A>", "course_id": "..."}, {"unit": "<unit B>", "course_id":
"..."}]}
```

- `blackout`: **Hard.** No course may meet in any slot that overlaps the window.
- `forbidden_timeslots`: **Hard.** The named course may not use the listed slots.
- `cross_list_match`: The two sections are one combined class taught across units, and they must end up in the same timeslot. You can't see the other unit's placement during `generate_schedule`. Placing the course at its preferred time is a good guess. The provost aligns the pair afterwards: it asks the non-owner first, through `evaluate_change` or an escalation.

TIMEOUTS AND FAILURES

Operation	Timeout	Notes
`generate_schedule`	180 s	Budget your LLM calls accordingly. A timeout counts as a failed response. On integration day the reference agent covers that one call so the exercise can continue, and the timeout still counts against your team. An explicit `status: "failed"` is recorded as your answer.
`evaluate_change`	60 s	
`repair_schedule`	180 s	

Each `request_id` is unique within a run. Exceptions raised inside `run_agent` are caught by the connector and reported as `status: "failed"`.

Error Handling Requirements

- Never invent a room or time slot that is absent from the coordinator input.
- Never silently ignore an unschedulable course.
- If a complete valid schedule is impossible, return status = partial and list the unscheduled courses in unresolved.
- If the request is malformed or your agent cannot operate, return status = failed and explain the error.
- Do not crash the integration process because one course cannot be scheduled.

The Provost Orchestrator NEW

The provost is the coordinator your agent works with, both locally and on integration day. Every run follows the same phases:

Phase	What happens
1 · Generate	All 11 units receive `generate_schedule` in parallel.
2 · Validate	Every returned schedule is checked against the unit's envelope and the interface schema. Violating entries are dropped and become unresolved.
3 · Negotiate	The provost merges the schedules and finds conflicts *between* units: shared rooms, shared instructors and cross-list mismatches. For each conflict it proposes a conflict-free move to one side through `evaluate_change`. It asks the unit that has conceded least so far first. On rejection it tries your alternatives, then the other unit. If no single move works, it escalates with `repair_schedule`.
4 · Disrupt	Scripted disruptions are applied. Each affected unit receives `repair_schedule`, and then negotiation runs again.
5 · Report	The final university schedule, per-unit metrics and the full message log are written to `outputs/<run-id>/`.

The provost holds the authoritative university schedule. Units propose; the provost commits. After your unit concedes a resource, the provost records the other unit's claim in your envelope as `unavailable_times`, so your later repairs don't move back into it.

CONNECTING YOUR AGENT

Your agent does **not** run a server. The connector dials out to the provost, waits for requests, calls your `run_agent`, and sends back what it returns. Only outbound network access is needed.

```
# practice: provost running in Docker on your own laptop
python student_kit/connect.py --unit SENG --agent path/to/agent.py --team "Team Name"

# final integration: the instructor's provost on the class network
python student_kit/connect.py --unit SENG --agent path/to/agent.py --team "Team Name" \
    --provost http://<instructor-ip>:8370
```

PRACTICE ENVIRONMENT

1. Install Docker Desktop and Python 3.9 or newer, then clone the course repository.
2. Start the provost with `docker compose up` (it pulls a prebuilt image) and open `http://localhost:8370`. That page is the live dashboard.
3. Connect your agent with the command above, then click **Start run**. A reference agent plays the other 10 units, and the practice data is the same shape as the final data.
4. Read your results on the dashboard and in `outputs/<run-id>/`. That folder contains `final_schedule.csv`, `metrics.json`, `moves.json`, and `messages.jsonl` (every request and response).

Useful tools:

- `connect.py envelope --unit SENG --out envelope.json` downloads your unit's envelope.
- `--save-dir captured/` records every request and response pair, which you can turn into test cases.
- `student_kit/check_response.py` runs the exact interface checks the provost applies. Use it in your tests.
- Hard-constraint checking is part of your agent and is not provided. During a run, the provost logs every violation it finds in your returned schedules to `messages.jsonl` as `"type": "hard_constraint_check"`.

INPUT SAMPLES

The `input_samples/` folder contains twelve real requests captured from the provost, each paired with an example of a schema-compliant response. They cover:

- `generate_schedule`
- `evaluate_change` for each of the three conflict types, plus one rejection with alternatives
- `repair_schedule` for each disruption type
- both kinds of escalation

Start your test suite from these samples.

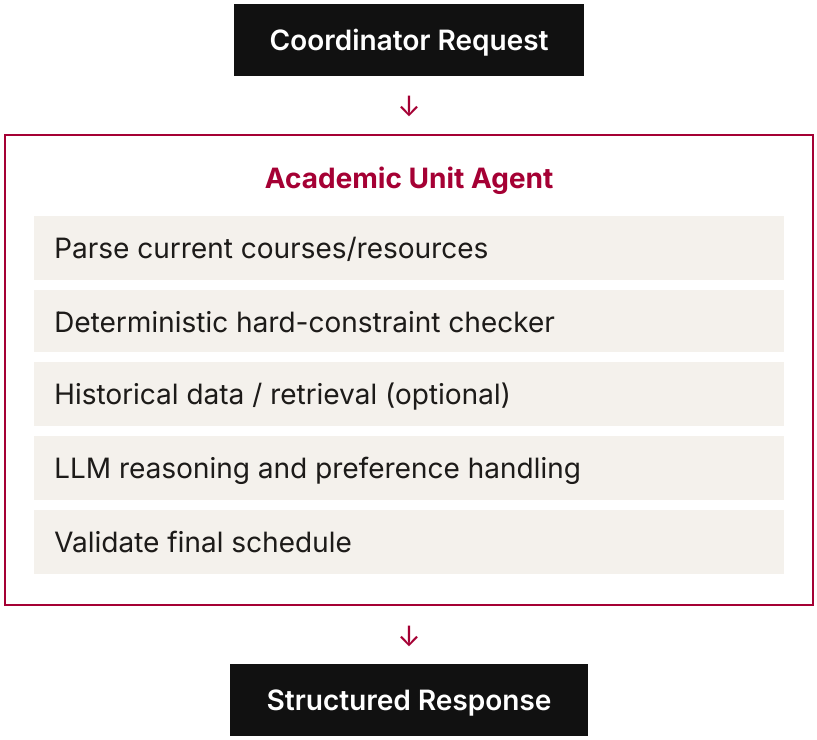
Agent Design Requirements

Your agent must include all of the following.

Requirement	Expectation
LLM-based reasoning	An LLM must play a substantive role in scheduling, evaluating changes, explaining decisions, or coordinating tools.
Explicit constraint handling	Your system must have a reliable method for checking hard constraints. Purely asking an LLM to 'be careful' is not enough.
Structured output	All coordinator responses must conform to the JSON-compatible interface in this specification.
Rationale	The agent must explain important scheduling decisions in concise, specific terms.
Alternative generation	When rejecting a feasible-to-repair change, the agent should propose alternatives rather than only saying no.
Historical-data decision	Document whether and how you use historical schedule data.
Evaluation	Test your agent systematically before final integration.

Recommended Architecture

A strong solution will normally separate deterministic constraint checking from LLM judgment. For example:



This is a recommendation, not a required implementation. The critical requirement is that hard constraints are checked reliably.

Testing and Evaluation UPDATED

Each team must create at least 12 test cases. At least four must test generate_schedule, four must test evaluate_change, and four must test repair_schedule.

- At least 3 tests must intentionally create difficult or impossible conditions.
- At least 2 tests must check room capacity or room-feature constraints.
- At least 2 tests must check faculty or course conflicts.
- At least 2 tests must evaluate whether the agent proposes sensible alternatives.
- At least 1 test must use a disruption that should be repairable with only one or two changes.

Requests from `input_samples/` or captured with `--save-dir` are good starting points. Modify them to create your hard cases.

REQUIRED EVALUATION METRICS

The provost computes the same metrics live during practice and integration runs.

Metric	Definition, and how the provost measures it	Target
Hard-constraint compliance	Percentage of completed test schedules containing zero hard violations. The provost re-checks every schedule you return.	100%
Task completion	Percentage of test cases for which the agent returns a usable response: status completed or partial, with a result object.	At least 90%
Interface compliance	Percentage of responses that conform to the required schema, using the checks in `student_kit/check_response.py`.	100%
Alternative validity	Percentage of proposed alternatives that are actually feasible within your envelope. The share that is also free university-wide is reported separately.	At least 90%
Repair stability	Average number of unnecessary course changes during repair: courses changed minus courses whose placement became infeasible.	Lower is better

Project Milestones UPDATED

Milestone 1	**Unit Model:** Course representation, constraints, preferences, historical-data plan, and proposed agent architecture.
Milestone 2	**Standalone Agent:** Agent can successfully complete generate_schedule for a supplied practice case, such as `input_samples/01` or your unit's practice envelope.
Milestone 3	**Change Negotiation:** Agent supports evaluate_change and proposes valid alternatives.
Milestone 4	**Repair and Evaluation:** Agent supports repair_schedule and has a completed 12-case test suite with metrics.
Milestone 5	**Integration Readiness:** Agent completes a full practice run against the provost container through `connect.py`, with 100% interface compliance on the dashboard. Submit that run's `metrics.json`.
Final	**Final Integration:** All 11 agents are connected to the instructor's provost and participate in a university-wide scheduling and disruption exercise.

Final Integration Exercise UPDATED

On final integration day, every team connects to the instructor's provost on the class network (address announced in class). The coordinator will provide each unit with its allocated rooms and time slots and request a schedule. The coordinator will then identify university-wide issues and ask individual academic-unit agents to evaluate changes. After the initial schedule converges, one or more disruptions will be introduced.

Possible disruptions include:

- A classroom becomes unavailable.
- A room's capacity changes.
- A faculty member becomes unavailable during a previously valid time.
- Enrollment in a course increases.
- A new university-wide blackout period is introduced.
- A course must be moved with minimal disruption to the rest of the schedule.

The final envelopes use the full section list for each unit, which is larger than the practice subset. Make sure your agent finishes within the timeouts at that scale. Keep your connector running until the instructor ends the run.

Your grade does not depend on the entire class-wide system producing a perfect university schedule. You are graded primarily on the quality, reliability, interface compliance, and evaluation of your academic-unit agent.

Required Deliverables

- `agent.py` exposing `run_agent(request: dict) -> dict`. It must run under `student_kit/connect.py`.
- All prompts, configuration files, and supporting code needed to run the agent, plus a `requirements.txt`.
- Any historical data or derived data used by the agent, or clear instructions for accessing it.
- README.md with setup and execution instructions.
- agent_design.pdf or agent_design.md (2–4 pages) describing architecture, constraints, use of historical data, and major design decisions.
- tests/ containing at least 12 documented test cases.
- evaluation.csv or evaluation.json containing your test results and required metrics, plus the `metrics.json` from your Milestone 5 practice run.
- A short final reflection describing one failure mode you discovered and how you addressed it.

SUBMISSION DIRECTORY

```
your_unit_name/
  agent.py
  README.md
  requirements.txt
  prompts/
  config/
  data/
  tests/
  agent_design.pdf (or .md)
  evaluation.csv (or .json)
```

Grading

Component	Percent	What matters
Academic-unit model and constraints	15%	Accurate representation of courses, resources, hard constraints, and meaningful soft preferences.
Agent design and LLM use	15%	Clear role for the LLM; appropriate architecture; design is justified rather than unnecessarily complex.
Schedule generation	15%	Produces feasible local schedules and handles unschedulable cases correctly.
Change evaluation and negotiation	15%	Accepts/rejects changes correctly, explains impacts, and proposes useful alternatives.
Repair behavior	10%	Repairs disruptions while preserving as much of the existing schedule as practical.
Testing and evaluation	15%	Thoughtful test cases, quantitative metrics, and evidence of iteration.
Interface and integration compliance	10%	Exact adherence to the common interface and successful coordinator integration.
Documentation and final explanation	5%	Clear README/design explanation and concise reflection on limitations.

Important Design Principles

- A more complex agent is not automatically a better agent.

- Hard constraints must be validated, not merely described in a prompt.
- Historical schedules are evidence of prior practice, not authoritative requirements.
- A scheduling agent should be able to explain why it made important choices.
- When a request cannot be satisfied, a useful agent explains the problem and offers alternatives.
- The final integrated system is multi-agent because scheduling knowledge and decision authority are distributed across academic units.

Academic Integrity and External Services UPDATED

You may use contemporary LLM APIs, agent frameworks, coding assistants, and open-source libraries. You are responsible for understanding and being able to explain your submitted system. Do not place confidential or non-public university information into external services. Use only data you are authorized to use. If you use historical schedules, document their source.

The envelopes and samples in the course repository have been prepared for this purpose. Instructor identities are pseudonymized, so you may send this data to external LLM APIs. Do not attempt to re-identify instructors.